МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе №8

по дисциплине «Организация процессов и программирование в среде Linux»

Тема: Взаимодействие процессов на основе сообщений

Студент гр. 3311 _____ Баймухамедов Р. Р.

Преподаватель _____ Лазарев Э. А.

Санкт-Петербург

2026

ЦЕЛЬ РАБОТЫ

Познакомиться с механизмом обмена сообщениями в Linux и системными вызовами приема и передачи сообщений. Реализовать согласование чтения общего файла тремя параллельными процессами с использованием общей очереди запросов и локальных очередей ответов.

ЗАДАНИЕ

1. Написать три программы, выполняющиеся параллельно и читающие один и тот же текстовый файл. Перед чтением каждая программа должна передать двум другим запросы через одну общую очередь сообщений и получить оба разрешения в своей локальной очереди.

2. В запрос включить номер программы-получателя, идентификатор очереди для ответа и время отправки запроса. В ответ включить номер отвечающей программы и время ответа.

3. Если программа уже прочитала файл, ответить сразу. Если чтение еще не выполнено, ответить на более ранний запрос. Остальные запросы запомнить и обслужить после чтения файла.

4. Перед выводом файла вывести номер программы и времена ответов от двух других программ. Откомпилировать программы и выполнить несколько запусков в разных терминалах и в различной последовательности.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Очередь сообщений System V – объект ядра, через который процессы передают отдельные сообщения. Сообщение содержит положительный тип long и полезные данные. Функция msgget создает очередь или возвращает идентификатор существующей. В программе общая очередь имеет ключ, полученный функцией ftok из файла ipc.key. Каждая программа создает

локальную очередь с IPC_PRIVATE и передает ее идентификатор другим процессам в запросе.

Функция msgsnd отправляет сообщение, а msgrcv принимает его и удаляет из очереди. Размер сообщения задается без поля типа long. Типы сообщений в общей очереди равны номерам получателей 1, 2 и 3, поэтому каждая программа извлекает только адресованные ей сообщения. Ответы направляются в локальную очередь отправителя запроса.

Программа использует msgrcv с IPC_NOWAIT, чтобы поочередно проверять общую и локальную очереди. Отсутствие сообщения обозначается ошибкой ENOMSG и не считается аварией. Между проверками выполняется nanosleep на 10 мс. Такой цикл позволяет одновременно обрабатывать чужие запросы и ожидать собственные разрешения.

Порядок чтения определяется временем собственного запроса. Время измеряется функцией clock_gettime с CLOCK_MONOTONIC в наносекундах от общей для процессов точки отсчета в одной системе. Если времена совпали, приоритет получает программа с меньшим номером. Это дополнительное правило устраняет взаимное ожидание при одинаковых временах.

После получения двух ответов программа читает файл, обслуживает отложенные запросы и сообщает остальным о завершении чтения. До завершения чтения всех участников она продолжает отвечать на новые запросы. Функция msgctl с IPC_RMID удаляет очередь. Общая очередь удаляется программой 1 только после подтверждений завершения от программ 2 и 3.

2 ПРОГРАММА

2.1 Подготовка входного файла

Для проверки используется файл input.txt с тремя строками. Во всех терминалах задаются одинаковые пути к входному файлу и файлу ipc.key. Файл ipc.key служит для получения ключа общей очереди и не содержит передаваемых сообщений.

```
user@linuxlabs:~/lab08$ touch ipc.key
user@linuxlabs:~/lab08$ printf 'First line \nSecond line \nThird line\n' > input.txt
user@linuxlabs:~/lab08$ cat -n input.txt
     1  First line
     2  Second line
     3  Third line
user@linuxlabs:~/lab08$
```

Рисунок 1 – Содержимое входного файла

2.2 Общая реализация обмена сообщениями

Общий алгоритм вынесен в файл reader_common.h. Три исходных файла reader1.c, reader2.c и reader3.c задают разные номера PROGRAM_ID и подключают этот файл. При компиляции образуются три отдельные программы с одинаковыми правилами взаимодействия.

Структура packet содержит вид сообщения kind, отправителя from, получателя to, идентификатор локальной очереди reply_q и время time_ns. REQUEST обозначает запрос, REPLY – разрешение на чтение, DONE – завершение чтения, FINISHED – завершение обслуживания очередей. Последние два вида сообщений используются для согласованного освобождения ресурсов.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <time.h>
#include <unistd.h>

enum { REQUEST = 1, REPLY, DONE, FINISHED };
```

```
struct packet {
    int kind, from, to, reply_q;
    uint64_t time_ns;
};
struct message {
    long type;
    struct packet data;
};

static int local_q = -1, common_q = -1;

static uint64_t now_ns(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) == -1) {
        perror("clock_gettime");
        exit(EXIT_FAILURE);
    }
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

static void cleanup(void)
{
    if (local_q != -1)
        msgctl(local_q, IPC_RMID, NULL);
}

static void fail(const char *where)
{
    perror(where);
    exit(EXIT_FAILURE);
}

static void send_packet(int queue, long type, int kind,
                        int to, uint64_t time_ns)
{
    struct message msg = {
        .type = type,
        .data = {kind, PROGRAM_ID, to, local_q, time_ns}
    };
    while (msgsnd(queue, &msg, sizeof(msg.data), 0) == -1) {
        if (errno != EINTR)
            fail("msgsnd");
    }
}

static int receive(int queue, long type, struct packet *data)
{
    struct message msg;
    ssize_t size = msgrcv(queue, &msg, sizeof(msg.data),
                          type, IPC_NOWAIT);
    if (size == -1) {
        if (errno == ENOMSG || errno == EINTR)
            return 0;
        fail("msgrcv");
    }
    if (size != (ssize_t)sizeof(msg.data)) {
        errno = EPROTO;
        fail("message size");
    }
    *data = msg.data;
```

```
    return 1;
}

static void show_file(const char *path, uint64_t replies[4])
{
    FILE *file = fopen(path, "r");
    if (file == NULL)
        fail("fopen");
    printf("Program %d: replies received\n", PROGRAM_ID);
    for (int peer = 1; peer <= 3; ++peer) {
        if (peer != PROGRAM_ID)
            printf("From %d: %llu ns\n", peer,
                   (unsigned long long)replies[peer]);
    }
    printf("Program %d: BEGIN FILE\n", PROGRAM_ID);
    int ch;
    while ((ch = fgetc(file)) != EOF) {
        if (putchar(ch) == EOF)
            fail("putchar");
    }
    if (ferror(file))
        fail("fgetc");
    if (fclose(file) == EOF)
        fail("fclose");
    printf("\nProgram %d: END FILE\n", PROGRAM_ID);
    if (fflush(stdout) == EOF)
        fail("fflush");
}

int main(int argc, char **argv)
{
    if (argc != 3) {
        fprintf(stderr, "Usage: %s input.txt ipc.key\n", argv[0]);
        return EXIT_FAILURE;
    }
    setbuf(stdout, NULL);
    key_t key = ftok(argv[2], 'R');
    if (key == (key_t)-1)
        fail("ftok");
    common_q = msgget(key, IPC_CREAT | 0600);
    if (common_q == -1)
        fail("msgget common");
    local_q = msgget(IPC_PRIVATE, 0600);
    if (local_q == -1)
        fail("msgget local");
    if (atexit(cleanup) != 0) {
        cleanup();
        return EXIT_FAILURE;
    }

    int peer_q[4] = {-1, -1, -1, -1};
    int deferred[4] = {0}, done[4] = {0};
    int finished[4] = {0}, got_reply[4] = {0};
    uint64_t replies[4] = {0};
    uint64_t request_time = now_ns();
    int have_read = 0, reply_count = 0;
    printf("Program %d: PID=%ld common=%d local=%d\n",
           PROGRAM_ID, (long)getpid(), common_q, local_q);
    printf("Request time: %llu ns\n",
           (unsigned long long)request_time);
    for (int peer = 1; peer <= 3; ++peer) {
```

```
    if (peer != PROGRAM_ID)
        send_packet(common_q, peer, REQUEST, peer, request_time);
}

for (;;) {
    struct packet msg;
    while (receive(common_q, PROGRAM_ID, &msg)) {
        int peer = msg.from;
        if (peer < 1 || peer > 3 || peer == PROGRAM_ID ||
            msg.to != PROGRAM_ID) {
            errno = EPROTO;
            fail("sender or recipient");
        }
        if (msg.kind == REQUEST) {
            peer_q[peer] = msg.reply_q;
            int earlier = msg.time_ns < request_time ||
                (msg.time_ns == request_time && peer < PROGRAM_ID);
            if (have_read || earlier) {
                send_packet(msg.reply_q, 1, REPLY, peer, now_ns());
                printf("Reply sent to %d\n", peer);
            } else {
                deferred[peer] = 1;
                printf("Request from %d deferred\n", peer);
            }
        } else if (msg.kind == DONE) {
            done[peer] = 1;
        } else {
            errno = EPROTO;
            fail("common message kind");
        }
    }
    while (receive(local_q, 1, &msg)) {
        int peer = msg.from;
        if (peer < 1 || peer > 3 || peer == PROGRAM_ID ||
            msg.to != PROGRAM_ID) {
            errno = EPROTO;
            fail("local sender or recipient");
        }
        if (msg.kind == REPLY && !got_reply[peer]) {
            got_reply[peer] = 1;
            replies[peer] = msg.time_ns;
            ++reply_count;
        } else if (msg.kind == FINISHED && PROGRAM_ID == 1) {
            finished[peer] = 1;
        }
    }
    if (!have_read && reply_count == 2) {
        show_file(argv[1], replies);
        have_read = 1;
        done[PROGRAM_ID] = 1;
        for (int peer = 1; peer <= 3; ++peer) {
            if (deferred[peer]) {
                send_packet(peer_q[peer], 1, REPLY, peer, now_ns());
                deferred[peer] = 0;
                printf("Deferred reply sent to %d\n", peer);
            }
            if (peer != PROGRAM_ID)
                send_packet(common_q, peer, DONE, peer, now_ns());
        }
    }
    if (done[1] && done[2] && done[3]) {
```

```
            if (PROGRAM_ID != 1) {
                send_packet(peer_q[1], 1, FINISHED, 1, now_ns());
                break;
            }
            if (finished[2] && finished[3]) {
                if (msgctl(common_q, IPC_RMID, NULL) == -1)
                    fail("msgctl common");
                puts("Common queue removed");
                break;
            }
        }
        if (now_ns() - request_time > 120000000000ULL) {
            fprintf(stderr, "Timeout: start all three programs\n");
            return EXIT_FAILURE;
        }
        struct timespec pause = {0, 10000000};
        nanosleep(&pause, NULL);
    }
    printf("Program %d: finished\n", PROGRAM_ID);
    return EXIT_SUCCESS;
}
```

2.3 Тексты трех программ

2.3.1 Файл reader1.c

```
#define PROGRAM_ID 1
#include "reader_common.h"
```

2.3.2 Файл reader2.c

```
#define PROGRAM_ID 2
#include "reader_common.h"
```

2.3.3 Файл reader3.c

```
#define PROGRAM_ID 3
#include "reader_common.h"
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММ

3.1 Компиляция

Каждый исходный файл компилируется отдельно. Дополнительные библиотеки и параметры линковки для системных вызовов очередей сообщений не требуются.

```
user@linuxlabs:~/lab08$ ls
reader1.c  reader2.c  reader3.c  reader_common.h
user@linuxlabs:~/lab08$ gcc reader1.c -o reader1
user@linuxlabs:~/lab08$ gcc reader2.c -o reader2
user@linuxlabs:~/lab08$ gcc reader3.c -o reader3
user@linuxlabs:~/lab08$ ls -la
total 88
drwxr-xr-x  2 user user  4096 Oct 13 17:12 .
drwxr-x--- 13 user user  4096 Oct 13 17:08 ..
-rwxrwxr-x  1 user user 17320 Oct 13 17:11 reader1
-rw-r--r--  1 user user    50 Oct 13 17:08 reader1.c
-rwxrwxr-x  1 user user 17280 Oct 13 17:12 reader2
-rw-r--r--  1 user user    50 Oct 13 17:08 reader2.c
-rwxrwxr-x  1 user user 17280 Oct 13 17:12 reader3
-rw-r--r--  1 user user    50 Oct 13 17:08 reader3.c
-rw-r--r--  1 user user  6696 Oct 13 17:08 reader_common.h
user@linuxlabs:~/lab08$ _
```

Рисунок 2 – Компиляция трех программ

3.2 Запуск в последовательности 1, 2, 3

Откроем три терминала и запустим программы в порядке 1, 2, 3. Первый процесс ожидает разрешения от двух других и не начинает чтение до их запуска. После завершения чтения он передает отложенные ответы, что позволяет следующему процессу получить оба разрешения.

```
user@linuxlabs:~/lab08$ ./reader1 input.txt ipc.key
Program 1: PID=2551 common=0 local=1
Request time: 8510305767547 ns
Request from 2 deferred
Request from 3 deferred
Program 1: replies received
From 2: 8512137877587 ns
From 3: 8513642491656 ns
Program 1: BEGIN FILE
First line
Second line
Third line

Program 1: END FILE
Deferred reply sent to 2
Deferred reply sent to 3
Common queue removed
Program 1: finished
user@linuxlabs:~/lab08$
```

Рисунок 3 – Работа программы 1 при порядке запуска 1, 2, 3

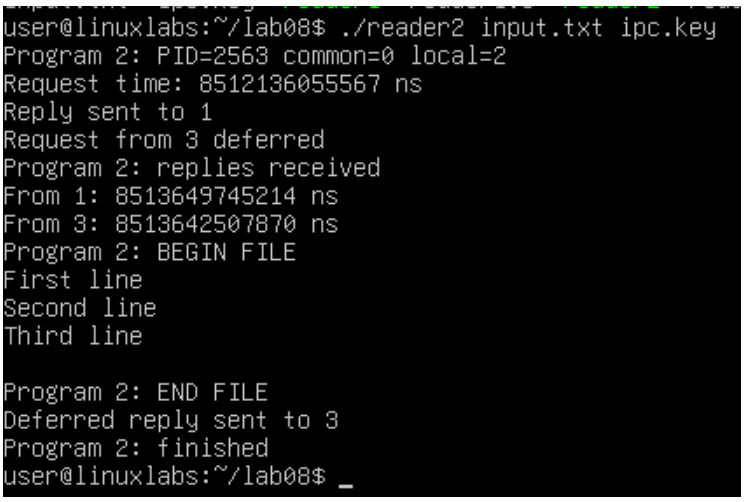


Рисунок 4 – Работа программы 2 при порядке запуска 1, 2, 3

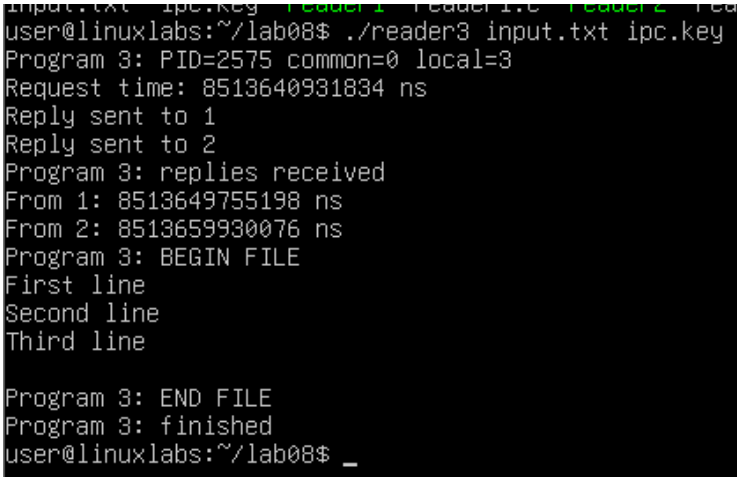


Рисунок 5 – Работа программы 3 при порядке запуска 1, 2, 3

3.3 Запуск в последовательности 3, 2, 1

Повторим опыт с обратным порядком запуска. Программа 3 формирует наиболее ранний запрос, получает разрешения от программ 2 и 1 и читает файл первой. Затем чтение выполняют программы 2 и 1.

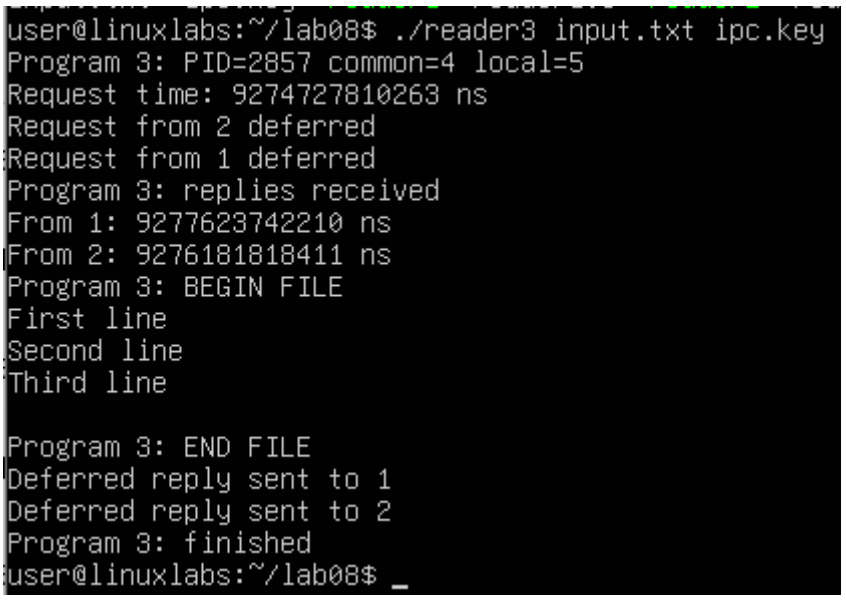


Рисунок 6 – Работа программы 3 при порядке запуска 3, 2, 1

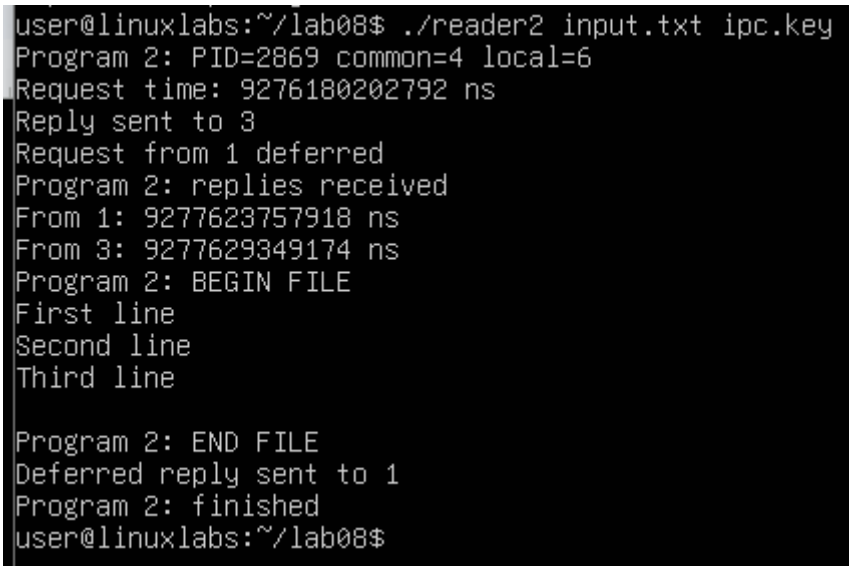


Рисунок 7 – Работа программы 2 при порядке запуска 3, 2, 1

```
user@linuxlabs:~/lab08$ ./reader1 input.txt ipc.key
Program 1: PID=2881 common=4 local=7
Request time: 9277621770978 ns
Reply sent to 3
Reply sent to 2
Program 1: replies received
From 2: 9277629558287 ns
From 3: 9277629339379 ns
Program 1: BEGIN FILE
First line
Second line
Third line

Program 1: END FILE
Common queue removed
Program 1: finished
user@linuxlabs:~/lab08$
```

Рисунок 8 – Работа программы 1 при порядке запуска 3, 2, 1

3.4 Запуск в последовательности 2, 1, 3

В третьем опыте первой запускается программа 2, затем программа 1 и программа 3. При таком порядке наиболее ранний запрос принадлежит программе 2. Остальные участники читают файл после получения обоих разрешений.

```
user@linuxlabs:~/lab08$ ./reader2 input.txt ipc.key
Program 2: PID=3032 common=8 local=9
Request time: 9376598821635 ns
Request from 1 deferred
Request from 3 deferred
Program 2: replies received
From 1: 9378299119493 ns
From 3: 9380274624537 ns
Program 2: BEGIN FILE
First line
Second line
Third line

Program 2: END FILE
Deferred reply sent to 1
Deferred reply sent to 3
Program 2: finished
user@linuxlabs:~/lab08$
```

Рисунок 9 – Работа программы 2 при порядке запуска 2, 1, 3

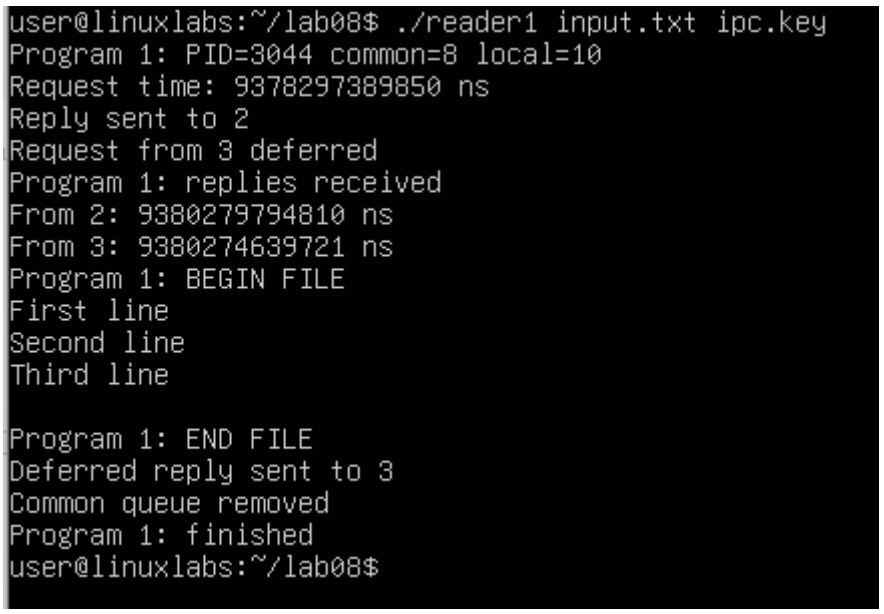


Рисунок 10 – Работа программы 1 при порядке запуска 2, 1, 3

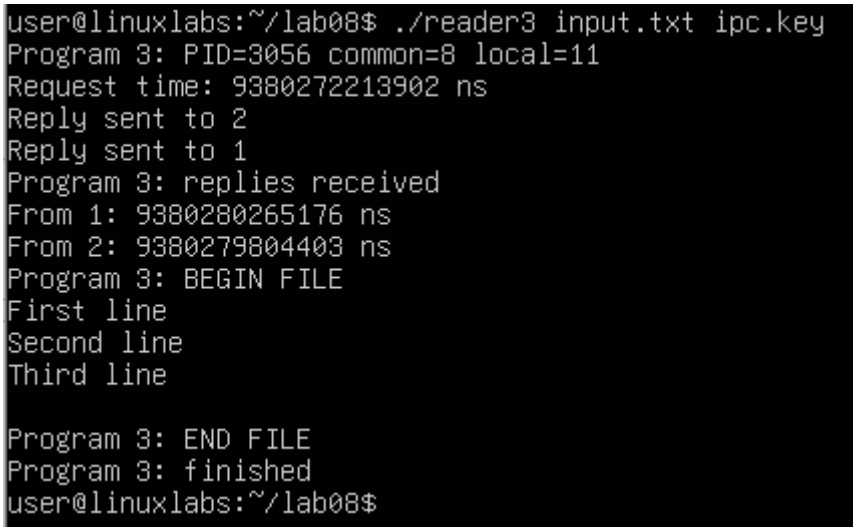


Рисунок 11 – Работа программы 3 при порядке запуска 2, 1, 3

3.5 Проверка удаления очередей

Каждая программа удаляет собственную локальную очередь при нормальном завершении. Программа 1 удаляет общую очередь после получения сообщений FINISHED от двух других участников и выводит Common queue removed.

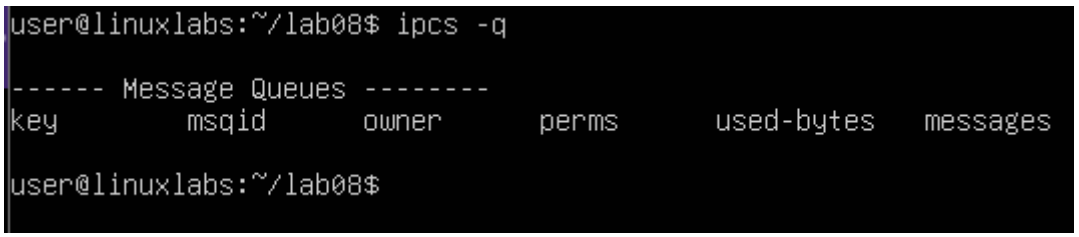


Рисунок 12 – Состояние очередей после завершения программ

4 АНАЛИЗ РЕЗУЛЬТАТОВ

До строки BEGIN FILE каждая программа выводит свой номер и две строки From с номерами ответивших участников и временем их ответов. Это позволяет проверить выполнение условия допуска к чтению. Между BEGIN FILE и END FILE отображается полное содержимое input.txt.

В первом опыте наиболее ранний запрос сформировала программа 1, во втором – программа 3, в третьем – программа 2. Времена запросов и сообщения об отложенных ответах подтверждают соответствующие порядки чтения 1, 2, 3, затем 3, 2, 1 и 2, 1, 3.

Выполнены три опыта с порядками запуска 1, 2, 3, затем 3, 2, 1 и 2, 1, 3. На скриншотах каждой программы представлены два полученных ответа и полный вывод трех строк входного файла. Все программы завершились, а программа 1 в каждом опыте вывела сообщение Common queue removed. После последнего опыта команда ipcs -q показала отсутствие очередей сообщений.

ВЫВОД

В ходе лабораторной работы изучены системные вызовы msgget, msgsnd, msgrcv и msgctl. Разработаны три программы, которые согласуют чтение общего файла через общую очередь запросов и отдельные очереди ответов. Чтение начинается после двух разрешений, отложенные запросы обслуживаются после чтения, а очереди удаляются после завершения взаимодействия.